

Activity 1 — Salary Analysis: Data Preprocessing

Group 08

2026-08-16

Contents

Read Data	2
1 Data Preprocessing	3
1.1 Remove the ID column	3
1.2 Check for missing values and empty strings	4
1.3 Remove duplicate records	4
1.4 Detect and handle outliers in <code>salary_in_usd</code>	4
1.5 Group granular job titles into major categories	6
1.6 Encode categorical variables	7
1.7 Scale numerical features	8
1.8 Final verification	8
1.9 Data dictionary	9
1.10 Save the cleaned dataset	10
2 Diagnostic plots (feeding Task 2 — EDA)	11
Distribution of salary (raw vs. log)	11
Salary by job category	12
Salary by experience level	13
Salary by company size	14
Salary by employment type	15
Salary by remote work ratio (discrete: 0/50/100)	16
Salary by company location group	17
Salary trend over time (work-year)	18
Number of jobs per category	19

Read Data

We start by loading the raw dataset `ds_salaries.csv` (607 observations, 11 variables) and inspecting its structure.

```
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.6.1
```

```
## Warning: package 'readr' was built under R version 4.6.1
```

```
## Warning: package 'forcats' was built under R version 4.6.1
```

```
## Warning: package 'lubridate' was built under R version 4.6.1
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v dplyr      1.2.1      v readr      2.2.0
```

```
## v forcats    1.0.1      v stringr    1.6.0
```

```
## v ggplot2    4.0.3      v tibble     3.3.1
```

```
## v lubridate  1.9.5      v tidyr      1.3.2
```

```
## v purrr      1.2.2
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
data <- read_csv("dataset/ds_salaries.csv")
```

```
head(data)
```

X	work_year	experience_level	employment_type	job_title	salary	salary_currency	salary_in_usd	employee_residence	remote_ratio	company_location	company_size
0	2020	MI	FT	Data Scientist	70000	EUR	79833	DE	0	DE	L
1	2020	SE	FT	Machine Learning Scientist	260000	USD	260000	JP	0	JP	S
2	2020	SE	FT	Big Data Engineer	85000	GBP	109024	GB	50	GB	M
3	2020	MI	FT	Product Data Analyst	20000	USD	20000	HN	0	HN	S
4	2020	SE	FT	Machine Learning Engineer	150000	USD	150000	US	50	US	L
5	2020	EN	FT	Data Analyst	72000	USD	72000	US	100	US	L

```
summary(data)
```

```
##           X           work_year    experience_level    employment_type
## Min.      : 0.0    Min.      :2020    Length      :607    Length      :607
## 1st Qu.:151.5    1st Qu.:2021    N.unique   : 4    N.unique   : 4
## Median :303.0    Median :2022    N.blank    : 0    N.blank    : 0
## Mean   :303.0    Mean   :2021    Min.nchar  : 2    Min.nchar  : 2
## 3rd Qu.:454.5    3rd Qu.:2022    Max.nchar  : 2    Max.nchar  : 2
## Max.    :606.0    Max.    :2022
##      job_title      salary      salary_currency salary_in_usd
## Length   :607    Min.      : 4000    Length   :607    Min.      : 2859
## N.unique  : 50    1st Qu.: 70000    N.unique  : 17    1st Qu.: 62726
## N.blank   : 0    Median   : 115000    N.blank   : 0    Median :101570
## Min.nchar: 11    Mean    : 324000    Min.nchar: 3    Mean    :112298
## Max.nchar: 40    3rd Qu.: 165000    Max.nchar: 3    3rd Qu.:150000
## Max.      :30400000    Max.      :600000
## employee_residence remote_ratio    company_location    company_size
## Length   :607    Min.      : 0.00    Length   :607    Length   :607
## N.unique  : 57    1st Qu.: 50.00    N.unique  : 50    N.unique  : 3
## N.blank   : 0    Median   :100.00    N.blank   : 0    N.blank   : 0
## Min.nchar: 2    Mean    : 70.92    Min.nchar: 2    Min.nchar: 1
## Max.nchar: 2    3rd Qu.:100.00    Max.nchar: 2    Max.nchar: 1
## Max.      :100.00
```

1 Data Preprocessing

1.1 Remove the ID column

The first column X is only a row index (0–606). It carries no information, so we remove it to avoid confusing the models later.

```
data$X <- NULL
head(data)
```

work_year	experience_level	employment_type	job_title	salary	salary_currency	salary_in_usd	employee_residence	remote_ratio	company_location	company_size
2020	MI	FT	Data Scientist	70000	EUR	79833	DE	0	DE	L
2020	SE	FT	Machine Learning Scientist	260000	USD	260000	JP	0	JP	S
2020	SE	FT	Big Data Engineer	85000	GBP	109024	GB	50	GB	M
2020	MI	FT	Product Data Analyst	20000	USD	20000	HN	0	HN	S
2020	SE	FT	Machine Learning Engineer	150000	USD	150000	US	50	US	L
2020	EN	FT	Data Analyst	72000	USD	72000	US	100	US	L

1.2 Check for missing values and empty strings

```
missing_values <- colSums(is.na(data))  
print(missing_values) ### The dataset has no Na values
```

```
##      work_year  experience_level  employment_type  job_title  
##           0           0           0           0  
##      salary  salary_currency  salary_in_usd  employee_residence  
##           0           0           0           0  
##  remote_ratio  company_location  company_size  
##           0           0           0
```

```
empty_strings <- colSums(data == "")  
print(empty_strings) ### The dataset has no empty strings
```

```
##      work_year  experience_level  employment_type  job_title  
##           0           0           0           0  
##      salary  salary_currency  salary_in_usd  employee_residence  
##           0           0           0           0  
##  remote_ratio  company_location  company_size  
##           0           0           0
```

The dataset contains **no missing values** and **no empty strings**, so no imputation or deletion is needed.

1.3 Remove duplicate records

```
duplicate_rows <- data[duplicated(data), ]  
print(nrow(duplicate_rows)) ### 42 duplicated rows
```

```
## [1] 42
```

```
data <- data %>% distinct()  
cat("Rows after removing duplicates:", nrow(data), "\n") ### 607 -> 565
```

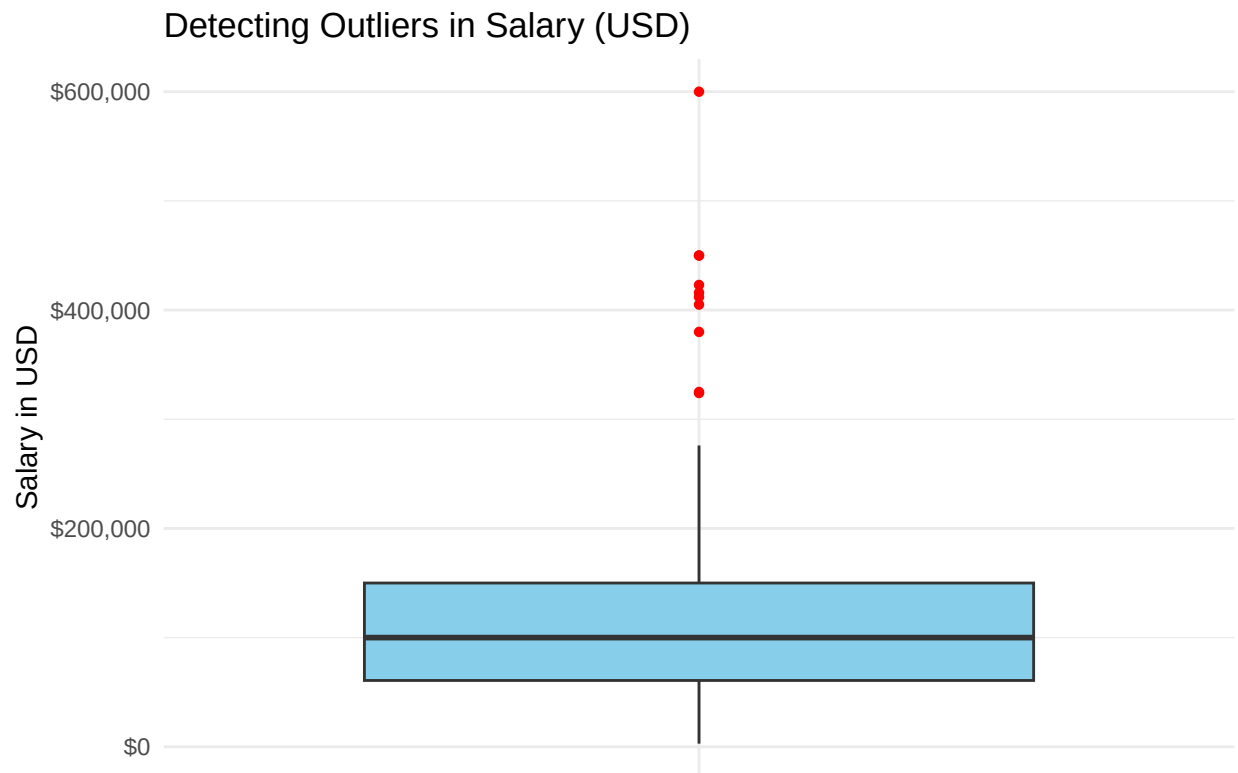
```
## Rows after removing duplicates: 565
```

42 rows are exact duplicates of other rows. We keep the first occurrence of each and remove the rest (**607** → **565**). Duplicates add no information and would double-count observations in every later statistic (means, ANOVA group sizes, model fitting).

1.4 Detect and handle outliers in salary_in_usd

Boxplot of the raw salary

```
ggplot(data, aes(x = "", y = salary_in_usd)) +
  geom_boxplot(fill = "skyblue", outlier.color = "red", outlier.shape = 16) +
  scale_y_continuous(labels = scales::dollar) +
  theme_minimal() +
  labs(title = "Detecting Outliers in Salary (USD)", x = "", y = "Salary in USD")
```



IQR rule (global)

```
Q1 <- quantile(data$salary_in_usd, 0.25)
Q3 <- quantile(data$salary_in_usd, 0.75)
IQR_value <- Q3 - Q1

lower_bound <- Q1 - 1.5 * IQR_value
upper_bound <- Q3 + 1.5 * IQR_value

outlier_rows <- data %>%
  filter(salary_in_usd < lower_bound | salary_in_usd > upper_bound)

cat("Total IQR Outliers Detected:", nrow(outlier_rows), "\n") # 10 outliers detected
```

```
## Total IQR Outliers Detected: 10
```

The IQR rule flags **10 observations** above 280,911 USD; none fall below the lower bound.

Decision: keep the outliers and log-transform

```
data$log_salary <- log(data$salary_in_usd) ### natural logarithm (ln)
```

We **keep all 10 rows**. They are legitimate executive/director salaries (EX/SE level, mostly US employers), not data-entry errors. Removing ~1.6% of an already small sample (565) would bias group means and group sizes, and winsorization would fabricate values. Instead, the skewness is handled at the model level by adding `log_salary = ln(salary_in_usd)`, which later tasks use as the response.

Cross-check: per-group IQR within experience level

A single global IQR ignores the fact that salary naturally varies by experience level. To verify that our flags are just high earners within each group (not data errors), we repeat the IQR rule **within each experience level**:

```
group_outliers <- data %>%
  group_by(experience_level) %>%
  mutate(gQ1 = quantile(salary_in_usd, 0.25),
         gQ3 = quantile(salary_in_usd, 0.75),
         gIQR = gQ3 - gQ1,
         is_group_outlier = salary_in_usd < (gQ1 - 1.5 * gIQR) |
                           salary_in_usd > (gQ3 + 1.5 * gIQR)) %>%
  ungroup()
print(table(group_outliers$experience_level, group_outliers$is_group_outlier))
```

```
##
##      FALSE TRUE
##  EN      86    2
##  EX      24    2
##  MI     203    5
##  SE     240    3
```

The flags are spread across all four levels (EN 2, MI 5, SE 3, EX 2), which confirms they are simply the top earners of each group — supporting our keep decision.

1.5 Group granular job titles into major categories

The dataset has 52 distinct job titles, too sparse for modeling. We group them into **7 major categories** using priority-ordered keyword matching. Management titles contain words like “Data Science” (e.g., “Head of Data Science”), so they must be matched **first**; `Research Scientist` must be matched before the generic `Scientist` pattern.

```
data <- data %>%
  mutate(job_category = case_when(
    grepl("Director|Head|Manager|Lead", job_title, ignore.case = TRUE) ~ "Management_Lead",
    grepl("Machine Learning|ML Engineer|ML Scientist|AI|Computer Vision|NLP|Deep Learning|Researcher",
          job_title, ignore.case = TRUE) ~ "ML_AI_Engineer",
    grepl("Research Scientist", job_title, ignore.case = TRUE) ~ "Research",
    grepl("Data Engineer|Big Data|ETL|Cloud Data|Data Architect|Data Specialist|Data Science Engineer",
          job_title, ignore.case = TRUE) ~ "Data_Engineer",
```

```

    grepl("Data Scientist|Scientist", job_title, ignore.case = TRUE) ~ "Data_Scientist",
    grepl("Data Analyst", job_title, ignore.case = TRUE) ~ "Data_Analyst",
    TRUE ~ "Other"
  ))

stopifnot(sum(is.na(data$job_category)) == 0)

data$job_category <- factor(data$job_category)
print(table(data$job_category))

##
##      Data_Analyst      Data_Engineer      Data_Scientist      Management_Lead      ML_AI_Engineer
##             101             151             143             58             81
##             Other             Research
##             15             16

```

Every title maps to exactly one category (the `stopifnot` assertion passes), and no title is left unmapped.

1.6 Encode categorical variables

Ordinal encoding

`experience_level` and `company_size` have a natural order, so we encode them as ordinal integers. The raw character columns are kept alongside for the Factorial ANOVA in Task 3.

```

data <- data %>%
  mutate(exp_level_ord = case_when(
    experience_level == "EN" ~ 1L,
    experience_level == "MI" ~ 2L,
    experience_level == "SE" ~ 3L,
    experience_level == "EX" ~ 4L
  ))

data <- data %>%
  mutate(company_size_ord = case_when(
    company_size == "S" ~ 1L,
    company_size == "M" ~ 2L,
    company_size == "L" ~ 3L
  ))

```

One-hot encoding with explicit reference levels

We use one-hot encoding (drop the reference level) for the nominal variables. The reference is set **explicitly** — never left to `factor()`'s alphabetical order, which would silently pick e.g. CT instead of FT for `employment_type`.

```

one_hot <- function(df, col, prefix, ref_level) {
  f <- relevel(factor(df[[col]]), ref = ref_level)
  mm <- as.data.frame(model.matrix(~ f - 1, data = df)[, -1, drop = FALSE])
  colnames(mm) <- paste0(prefix, gsub("[^[:alnum:]]", "_", levels(f)[-1]))
  mm
}

```

company_location has 53 countries, 36 of which have at most 3 rows. Keeping all of them would create ~52 near-empty dummies and unstable estimates, so we collapse to the **Top 7 countries + Other** (US alone is ~63% of the sample).

```
top_countries <- c("US", "GB", "CA", "DE", "IN", "FR", "ES")
data <- data %>%
  mutate(location_group = ifelse(company_location %in% top_countries,
                                company_location, "Other"),
         residence_group = ifelse(employee_residence %in% top_countries,
                                employee_residence, "Other"))

data <- bind_cols(
  data,
  one_hot(data, "job_category", "job_cat_", ref_level = "Data_Analyst"),
  one_hot(data, "employment_type", "empl_", ref_level = "FT"),
  one_hot(data, "location_group", "loc_", ref_level = "Other"),
  one_hot(data, "residence_group", "res_", ref_level = "Other")
)
```

employment_type is highly imbalanced (FT dominates; PT/CT/FL are rare). We keep all four levels and one-hot encode them; the imbalance is acknowledged in the report, and the ANOVA will use Type III sums of squares because of it.

For non-remote employees, the company location equals the employee residence, so the two dummy sets would be strongly correlated in a regression. We add a single binary is_local indicator as a low-collinearity alternative.

```
data$is_local <- as.integer(data$company_location == data$employee_residence)
```

1.7 Scale numerical features

Numerical predictors are standardized (z-score) because penalized models (Ridge/LASSO) require scale-invariant features. Raw columns are preserved for interpretable models (MLR, ANOVA).

```
data$salary_in_usd_scaled <- as.numeric(scale(data$salary_in_usd))
data$log_salary_scaled <- as.numeric(scale(data$log_salary))
data$remote_ratio_scaled <- as.numeric(scale(data$remote_ratio))
data$exp_level_ord_scaled <- as.numeric(scale(data$exp_level_ord))
data$company_size_ord_scaled <- as.numeric(scale(data$company_size_ord))
```

1.8 Final verification

```
cat("Rows after cleaning:", nrow(data), "\n")      ### 565
```

```
## Rows after cleaning: 565
```

```
cat("Missing values after cleaning:", sum(is.na(data)), "\n") ### 0
```

```
## Missing values after cleaning: 0
```



```
cat("Duplicates after cleaning:", sum(duplicated(data)), "\n")### 0
```

```
## Duplicates after cleaning: 0
```

Contingency tables for downstream planning

These tables are printed for Members B & C: the first checks whether the Job Category $\times$ Experience Level interaction has sparse cells; the second quantifies the overlap between the location and residence dummies.

```
print(table(data$job_category, data$experience_level))
```

```
##
##              EN EX MI SE
## Data_Analyst  17  3 38 43
## Data_Engineer  15  5 60 71
## Data_Scientist 23  1 62 57
## Management_Lead 0 14  9 35
## ML_AI_Engineer 23  0 30 28
## Other         6  3  2  4
## Research      4  0  7  5
```

```
print(table(data$location_group, data$residence_group))
```

```
##
##      CA  DE  ES  FR  GB  IN Other  US
## CA    26   0   0   0   1   0     0   1
## DE     0  22   0   1   0   0     4   0
## ES     0   0  13   1   0   0     0   0
## FR     0   0   0  14   0   0     0   1
## GB     0   0   0   0  42   0     4   0
## IN     0   0   0   0   0  24     0   0
## Other  0   1   1   1   0   2    88   0
## US     1   1   1   1   0   4    17 293
```

```
cat("is_local = 1 (location == residence):", sum(data$is_local),
    "of", nrow(data), "rows\n")
```

```
## is_local = 1 (location == residence): 514 of 565 rows
```

1.9 Data dictionary

```
data_dictionary <- data.frame(
  Variable = c("work_year", "experience_level", "employment_type", "job_title",
    "salary", "salary_currency", "salary_in_usd", "employee_residence",
    "remote_ratio", "company_location", "company_size", "log_salary",
    "job_category", "exp_level_ord", "company_size_ord",
    "location_group", "residence_group", "is_local", "job_cat_*",
    "empl_*", "loc_*", "res_*", " *_scaled"),
```

```

Type = c("numeric", "character", "character", "character", "numeric",
         "character", "numeric", "character", "numeric", "character",
         "character", "numeric", "factor", "ordinal (1-4)", "ordinal (1-3)",
         "factor", "factor", "binary (0/1)", "one-hot (6)", "one-hot (3)",
         "one-hot (7)", "one-hot (7)", "z-score"),
Meaning = c("Year the salary was paid", "Experience level (EN/MI/SE/EX)",
            "Employment type (PT/FT/CT/FL)", "Original granular job title",
            "Gross salary in original currency", "ISO 4217 currency code",
            "Salary converted to USD (response)", "Employee's country of residence",
            "Remote work ratio (0/50/100)", "Employer's country",
            "Company size (S/M/L)", "ln(salary_in_usd) for modeling",
            "Grouped job category (7 levels)", "Ordinal encoding of experience",
            "Ordinal encoding of company size", "Top-7 countries + Other",
            "Top-7 countries + Other (residence)",
            "1 if company_location == employee_residence",
            "Dummies of job_category (ref: Data_Analyst)",
            "Dummies of employment_type (ref: FT)", "Dummies of location_group (ref: Other)",
            "Dummies of residence_group (ref: Other)", "Standardized numeric features")
)
print(data_dictionary)

```

##	Variable	Type	Meaning
## 1	work_year	numeric	Year the salary was paid
## 2	experience_level	character	Experience level (EN/MI/SE/EX)
## 3	employment_type	character	Employment type (PT/FT/CT/FL)
## 4	job_title	character	Original granular job title
## 5	salary	numeric	Gross salary in original currency
## 6	salary_currency	character	ISO 4217 currency code
## 7	salary_in_usd	numeric	Salary converted to USD (response)
## 8	employee_residence	character	Employee's country of residence
## 9	remote_ratio	numeric	Remote work ratio (0/50/100)
## 10	company_location	character	Employer's country
## 11	company_size	character	Company size (S/M/L)
## 12	log_salary	numeric	ln(salary_in_usd) for modeling
## 13	job_category	factor	Grouped job category (7 levels)
## 14	exp_level_ord	ordinal (1-4)	Ordinal encoding of experience
## 15	company_size_ord	ordinal (1-3)	Ordinal encoding of company size
## 16	location_group	factor	Top-7 countries + Other
## 17	residence_group	factor	Top-7 countries + Other (residence)
## 18	is_local	binary (0/1)	1 if company_location == employee_residence
## 19	job_cat_*	one-hot (6)	Dummies of job_category (ref: Data_Analyst)
## 20	empl_*	one-hot (3)	Dummies of employment_type (ref: FT)
## 21	loc_*	one-hot (7)	Dummies of location_group (ref: Other)
## 22	res_*	one-hot (7)	Dummies of residence_group (ref: Other)
## 23	*_scaled	z-score	Standardized numeric features

1.10 Save the cleaned dataset

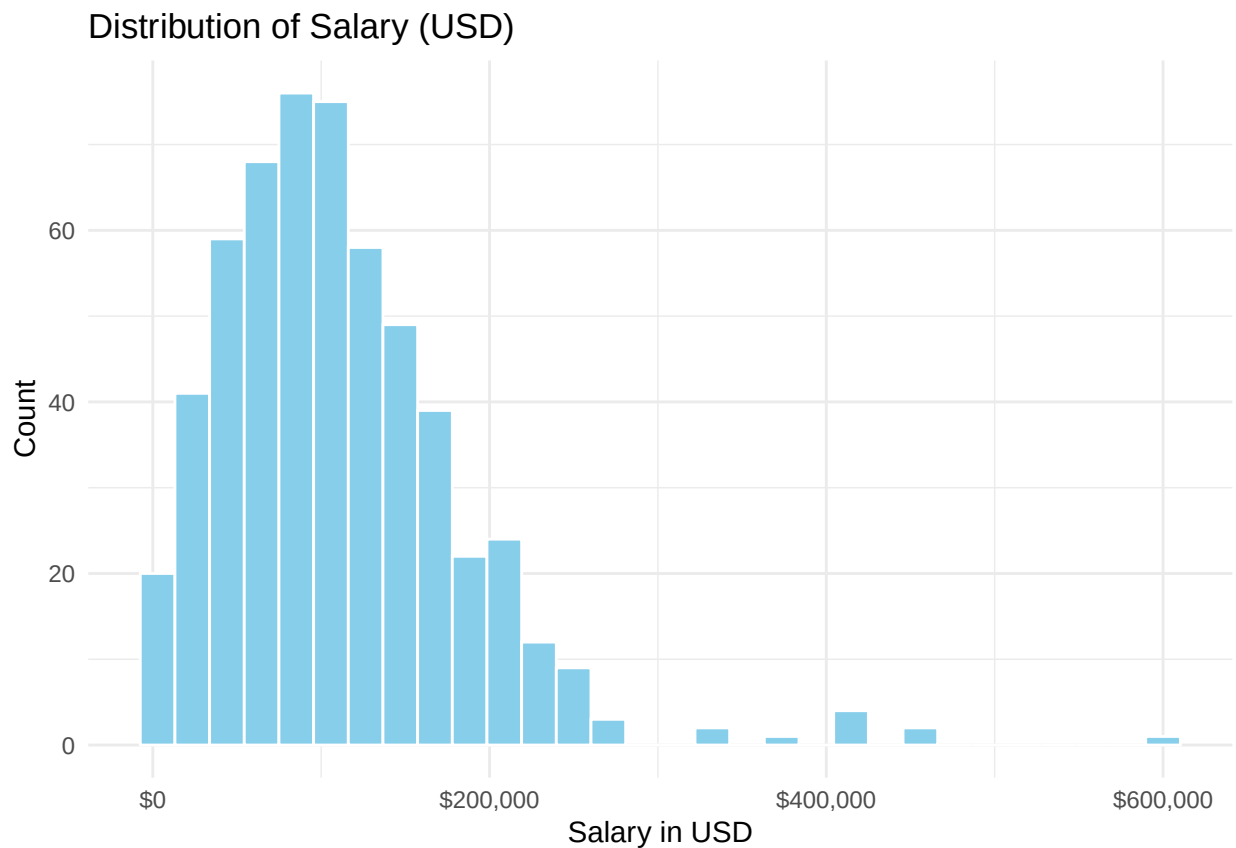
The cleaned and encoded dataset is written once and serves as the **single source of truth** for Tasks 3–5 (ANOVA, regression, logistic regression).

```
write.csv(data, "dataset/ds_salaries_clean.csv", row.names = FALSE)
```

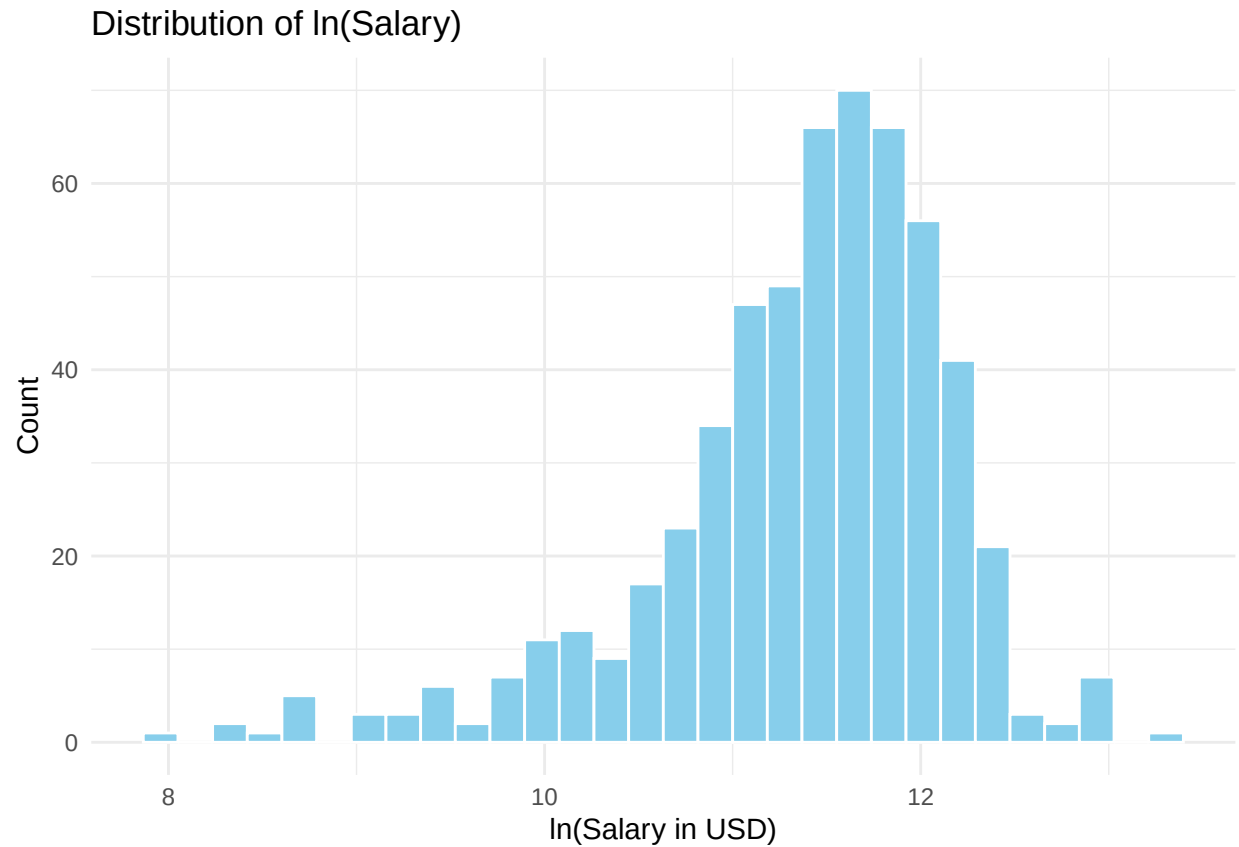
2 Diagnostic plots (feeding Task 2 — EDA)

Distribution of salary (raw vs. log)

```
ggplot(data, aes(x = salary_in_usd)) +  
  geom_histogram(fill = "skyblue", color = "white", bins = 30) +  
  scale_x_continuous(labels = scales::dollar) +  
  theme_minimal() +  
  labs(title = "Distribution of Salary (USD)", x =  
    "Salary in USD", y = "Count")
```

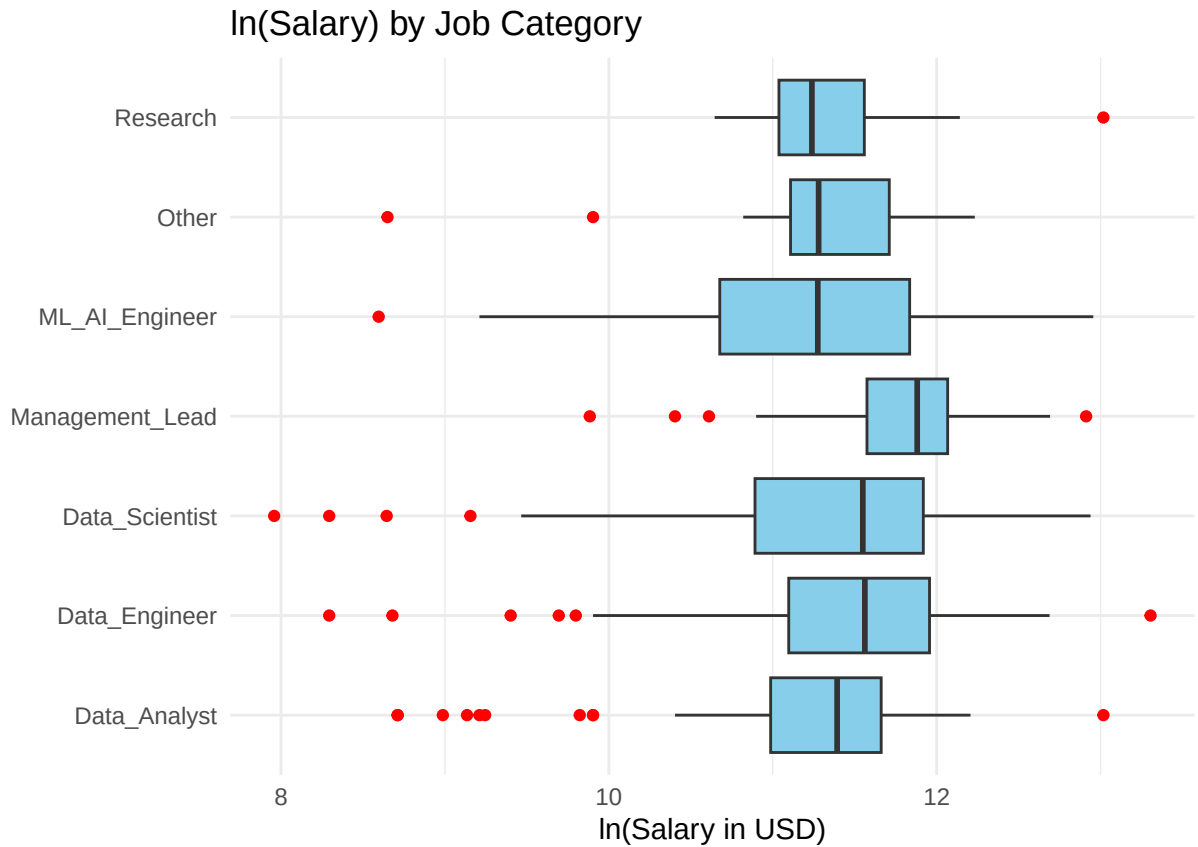


```
ggplot(data, aes(x = log_salary)) +  
  geom_histogram(fill = "skyblue", color = "white", bins = 30) +  
  theme_minimal() +  
  labs(title = "Distribution of ln(Salary)", x =  
    "ln(Salary in USD)", y = "Count")
```



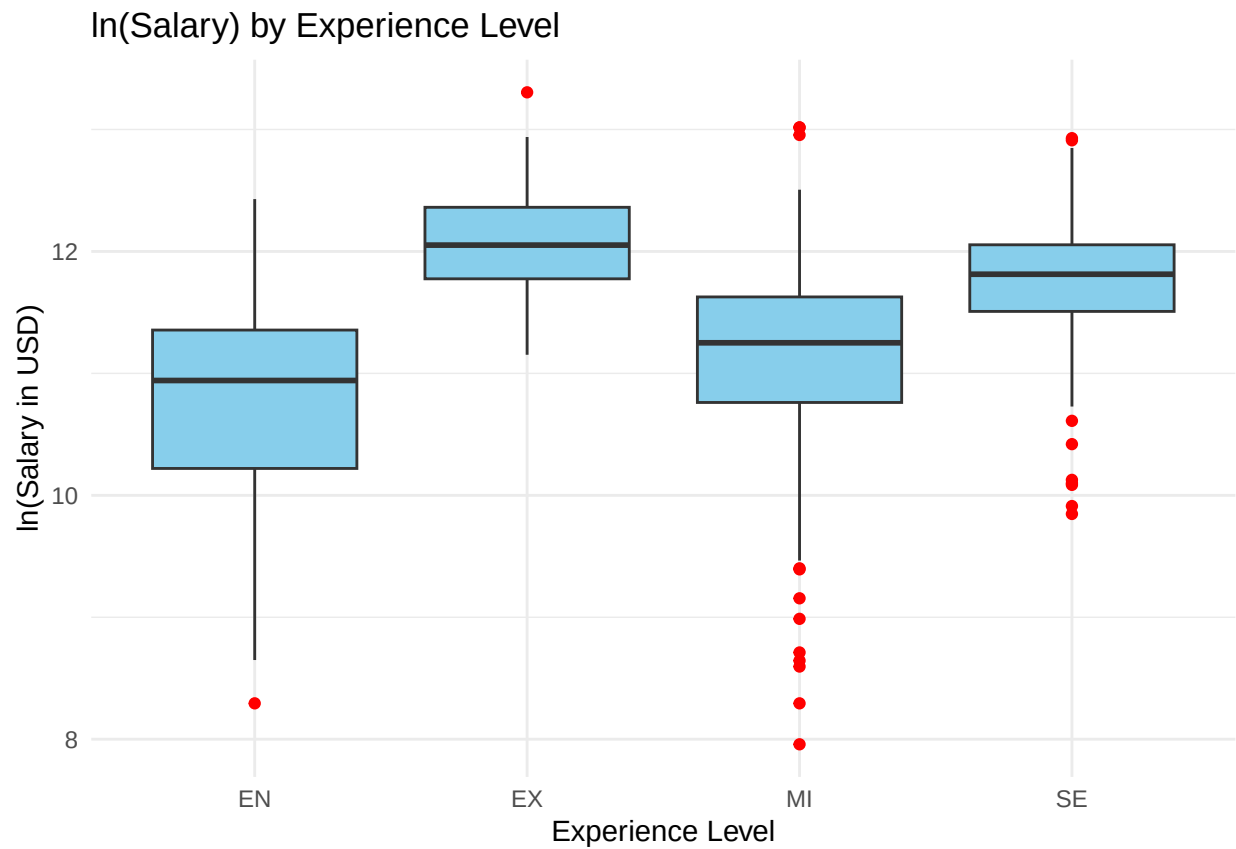
Salary by job category

```
ggplot(data, aes(x = job_category, y = log_salary)) +  
  geom_boxplot(fill = "skyblue", outlier.color = "red") +  
  coord_flip() +  
  theme_minimal() +  
  labs(title = "ln(Salary) by Job Category", x = "", y = "ln(Salary in USD)")
```



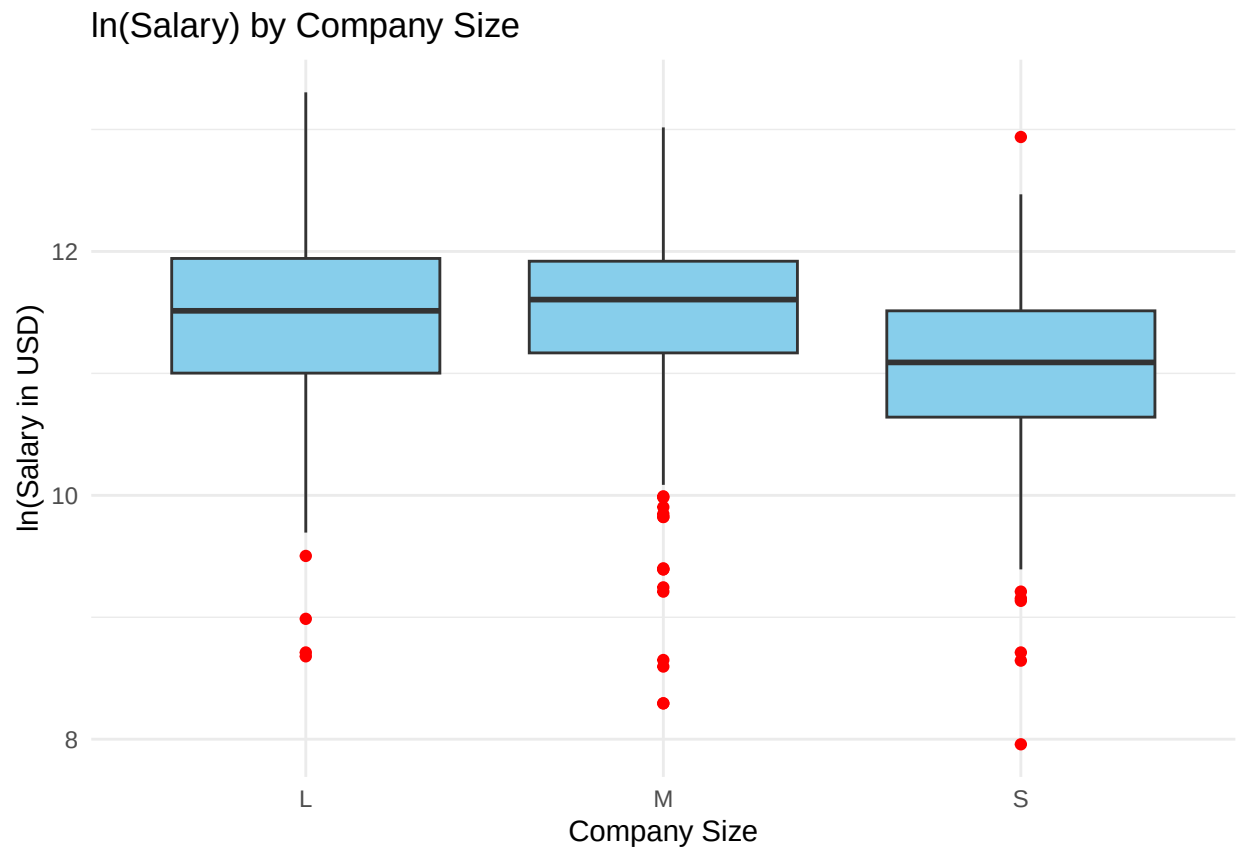
Salary by experience level

```
ggplot(data, aes(x = experience_level, y = log_salary)) +
  geom_boxplot(fill = "skyblue", outlier.color = "red") +
  theme_minimal() +
  labs(title = "ln(Salary) by Experience Level", x =
    "Experience Level", y = "ln(Salary in USD)")
```



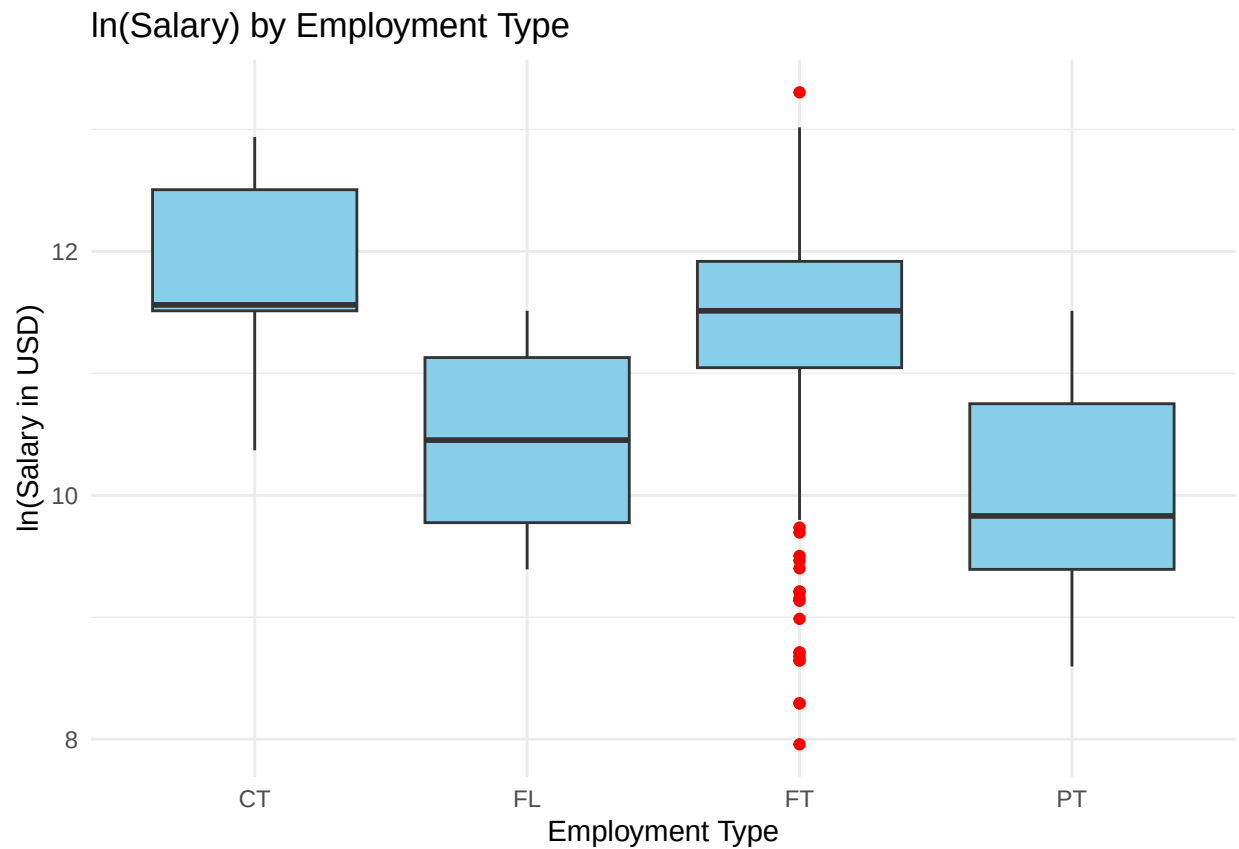
Salary by company size

```
ggplot(data, aes(x = company_size, y = log_salary)) +  
  geom_boxplot(fill = "skyblue", outlier.color = "red") +  
  theme_minimal() +  
  labs(title = "ln(Salary) by Company Size", x =  
        "Company Size", y = "ln(Salary in USD)")
```



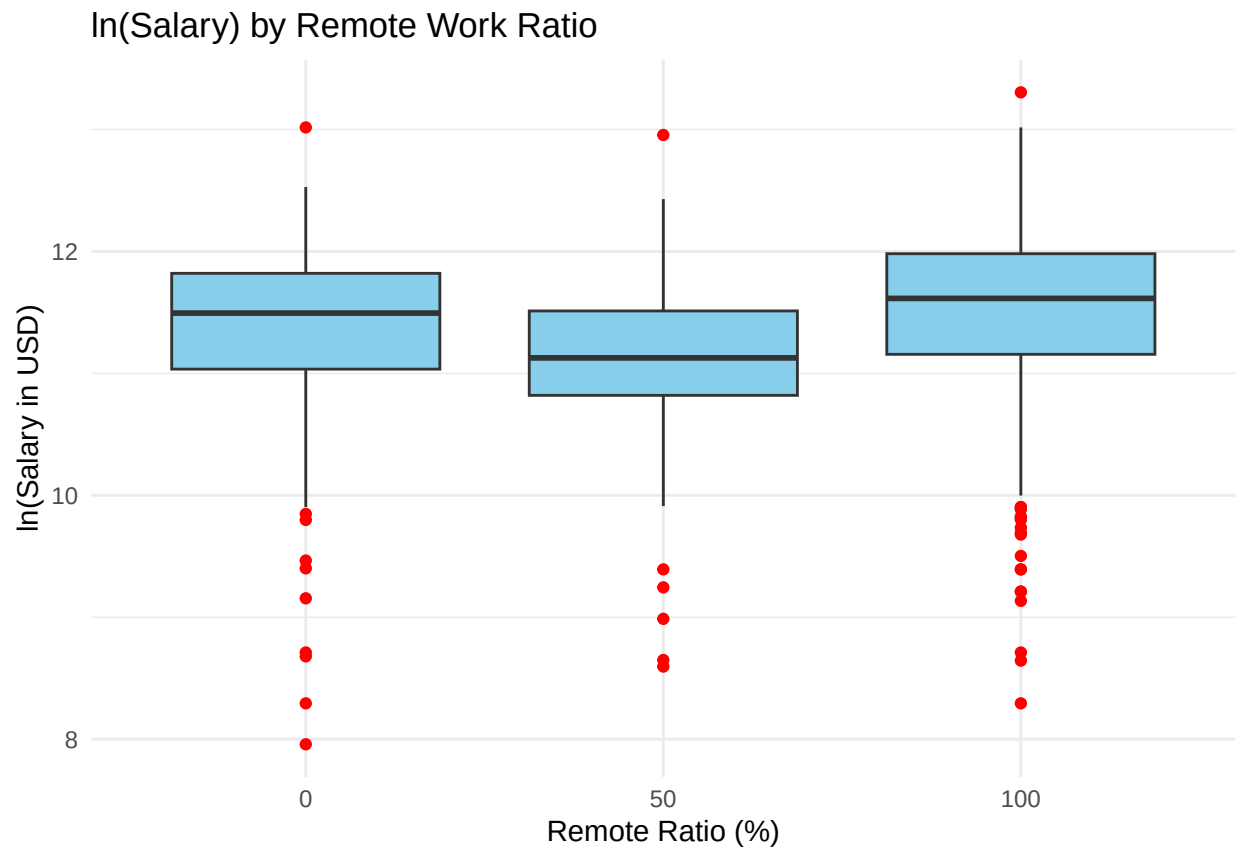
Salary by employment type

```
ggplot(data, aes(x = employment_type, y = log_salary)) +  
  geom_boxplot(fill = "skyblue", outlier.color = "red") +  
  theme_minimal() +  
  labs(title = "ln(Salary) by Employment Type", x =  
        "Employment Type", y = "ln(Salary in USD)")
```



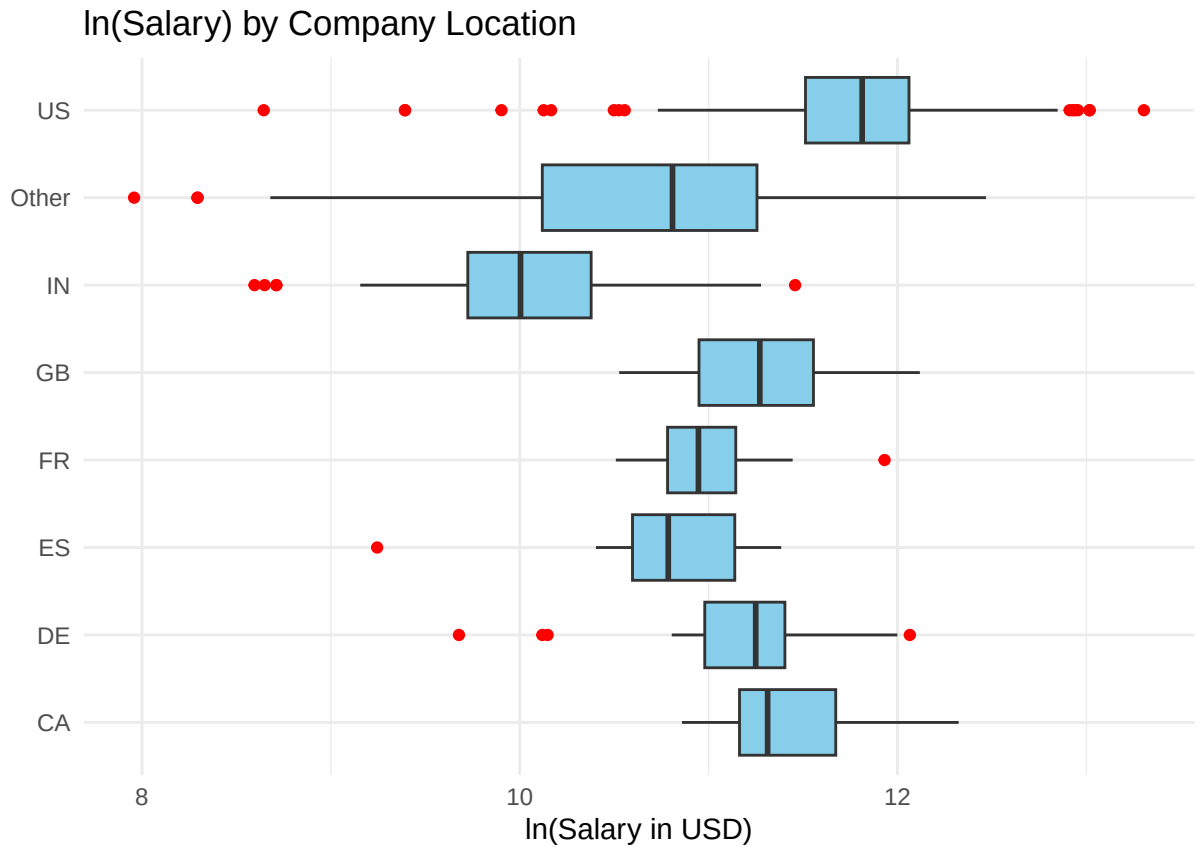
Salary by remote work ratio (discrete: 0/50/100)

```
ggplot(data, aes(x = factor(remote_ratio), y = log_salary)) +  
  geom_boxplot(fill = "skyblue", outlier.color = "red") +  
  theme_minimal() +  
  labs(title = "ln(Salary) by Remote Work Ratio", x =  
        "Remote Ratio (%)", y = "ln(Salary in USD)")
```

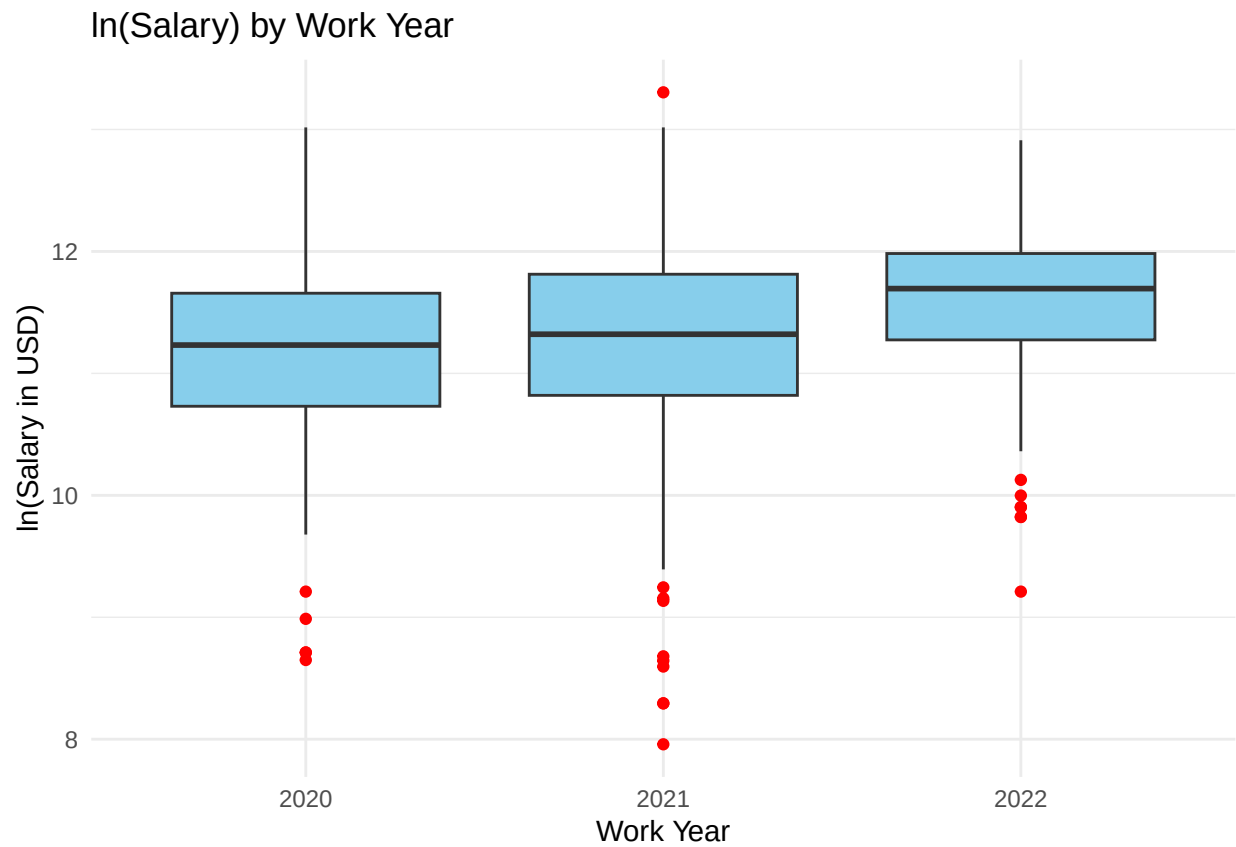
Salary by company location group

```
ggplot(data, aes(x = location_group, y = log_salary)) +  
  geom_boxplot(fill = "skyblue", outlier.color = "red") +  
  coord_flip() +  
  theme_minimal() +  
  labs(title = "ln(Salary) by Company Location", x =  
        "", y = "ln(Salary in USD)")
```



Salary trend over time (work-year)

```
ggplot(data, aes(x = factor(work_year), y = log_salary)) +
  geom_boxplot(fill = "skyblue", outlier.color = "red") +
  theme_minimal() +
  labs(title = "ln(Salary) by Work Year", x =
        "Work Year", y = "ln(Salary in USD)")
```



Number of jobs per category

```
ggplot(data, aes(x = job_category)) +  
  geom_bar(fill = "skyblue") +  
  coord_flip() +  
  theme_minimal() +  
  labs(title = "Number of Jobs per Category", x =  
        "", y = "Count")
```

